Static Plan or Dynamic Dispatch? A Measured Comparison of Scheduling DAG Workloads on a Hybrid CPU/GPU Laptop with One Cost Model and End-to-End Timing

Anonymous Authors

Affiliation withheld for double-blind review

***Abstract*—Schedulers for heterogeneous machines are mostly evaluated in simulation, rarely report their own cost, and are seldom compared fairly with the oldest alternative: a *static* plan computed before the run. We compare static plans (HEFT, PEFT and HEFT with end-to-end cost terms) with *dynamic* dispatch (blind greedy dispatch, a parameter-aware rule, and a predicted-rank dispatcher that binds a task only to an idle worker) on real executions of an image-processing task graph on a laptop with six performance cores, seven efficiency cores and one CUDA worker (RTX 3050). Every policy uses the same ridge cost model; GPU eligibility is taken per task from recorded speed-up measurements; the primary metric is the end-to-end makespan, from the start of the run to the last completion, so planning and start-up time are counted. On the primary workload (24 included blocks, ten Holm-adjusted contrasts) the dynamic dispatcher is 17.8% [16.7, 18.8] faster than blind dispatch on the CPU and 21.2% faster with a strict GPU set, and it is 5.2% [4.6, 5.9] (CPU) and 12.2% [11.1, 13.4] (strict GPU set) faster than the static HEFT plan built from the same model. Using the GPU only where a speed-up was measured helps the dispatcher by 2.6% to 7.5% and does not help blind dispatch. The claim has limits: a hand rule that knows the cost-relevant task parameter is within 1% of the dispatcher in every configuration; the policies tie when task cost does not vary; end-to-end cost terms change nothing (equivalent within ±2%); a hybrid selector would gain at most 1.1%; the GPU cost estimate has a median error of 23% to 31%; and an earlier session found HEFT level with the dispatcher. We release the protocol, the data and 521 tests.**

***Index Terms*—*task scheduling, static versus dynamic scheduling, HEFT, runtime prediction, CPU/GPU, hybrid cores, empirical evaluation, reproducibility*

I. Introduction

Modern machines combine core types and accelerators, and applications are task graphs whose edges are data dependencies. A scheduler decides in which *order* ready tasks are served and on which *worker* each one runs. Two designs are available. A *static* scheduler such as HEFT or PEFT [1], [2] computes a complete plan before the run from a cost estimate for every task on every worker. A *dynamic* scheduler decides when a worker becomes idle or a task becomes ready, as runtime systems such as StarPU [3] and most learned schedulers [4], [5], [6] do. The static plan pays a planning cost once and cannot react; the dynamic scheduler can react but pays a decision cost throughout and may bind work with less information.

Published comparisons rarely isolate this choice. Reported improvements of 11% to 27% [4], [7], [8], [6] come from simulators; none of the 12 papers we examined reports a confidence interval; measured decision time appears in few; and, to our knowledge, none gives the static and the dynamic scheduler the *same* cost model and measures both end to end, including the time a plan takes to compute. A simple question follows: **does dynamic dispatch finish a workflow sooner than a static plan built from the same model, on a real hybrid CPU+GPU machine, once planning, start-up and scheduling time are counted, and under which conditions?**

We answer it with an instrumented runtime on a laptop and make five contributions. (1) A reproducible end-to-end timing convention and benchmark that gives static and dynamic policies one cost model. (2) A fair set of static baselines (HEFT, PEFT, HEFT with end-to-end terms, an oracle reference) and a dynamic dispatcher with and without end-to-end cost terms. (3) Per-task GPU eligibility derived from recorded speed-up measurements, so that the GPU is used only where it was shown to help. (4) A pre-stated confirmatory family of ten contrasts with Holm correction, six workloads, and a calibration/hold-out/evaluation split. (5) An honest account of the limits: a parameter-aware rule that captures almost all of the gain, ties where task cost does not vary, a GPU model with large error, and a hybrid selector that is not justified. We claim neither a new learning algorithm nor superiority over learned schedulers, which we did not run.

II. Background and Related Work

Static list scheduling. HEFT ranks tasks by upward rank and assigns each, in rank order, to the processor with the earliest finish time, inserting it into idle gaps; PEFT replaces the rank by an optimistic cost table [1], [2]. Both need a cost matrix and are usually evaluated in simulation. **Learned DAG scheduling.** LACHESIS [4] combines a GNN task selector with a duplication-based placement rule; later work uses GCN or Transformer encoders with PPO [9], [6], or DQN, actor-critic and meta-RL formulations for edge offloading [10], [11], [12], [13], [14], [15], [5], [7], [8]. Only [16] (CPU+FPGA) and [5] (a three-board Jetson testbed) use physical devices.

Prediction-driven scheduling. [17] predicts task runtimes and drives HEFT in simulation; [18] report that, under high utilisation, order matters more than estimate accuracy; [19] place AI tools with an LLM agent on real CPU/GPU hardware

against HEFT and StarPU on small DAGs; StarPU [3] has long used history-based performance models. **Patents.** The base patent [20] claims task graphs, per-kernel queues and thread-based dispatch without prediction; IBM's grants [21], [22] disclose a learning agent that ranks DAG tasks and schedules with execution estimates, Samsung's [23] predicts durations on heterogeneous cores, and Intel's [24] profiles CPU/GPU throughput. **Products.** Operating systems place threads using hardware feedback or energy models [25], [26], [27]; cluster products schedule against declared resources [28], [29], [30], [31]; none of the cited product pages describes execution-time prediction in the scheduler.

TABLE I

PATENTS CLOSEST TO THE PROJECT (AS SUMMARISED FROM GOOGLE PATENTS; LEGAL STATUS IS AN ASSUMPTION, NOT AN OPINION)

Patent	Discloses	Prediction
[20] Xilinx, US 11,561,826 B1	task graph; kernel objects tied to compute circuits; per-kernel task queues; in-degree gating; performance-based thread adjustment (base work)	no
[21] IBM, US 11,740,933 B2	learning agent assigns a rank to DAG tasks; tasks run in rank order on a heterogeneous SoC	learned rank
[22] IBM, US 11,966,776 B2	learning agent schedules DAGs with priorities and task execution estimates, may drop DAGs	yes
[24] Intel, US 10,186,007 B2	CPU+GPU: profiling threads time part of the work on each device and split the rest	online profiling
[23] Samsung, US 10,162,671 B2	predicts a task's duration when core state changes, from solo-versus-concurrent ratios	yes

None of the patents summarised describes an empirical method for deciding when prediction is worth its cost; rank-ordered dispatch with a learning agent is disclosed by [21], so the contribution of this paper is the measured comparison, not the idea of ranking. A professional prior-art search is needed before any filing.

TABLE II

PRODUCTS THAT DECIDE WHERE WORK RUNS (ONLY WHAT THE CITED DOCUMENTATION PAGES STATE)

Product	Scope	Heterogeneity handled	Prediction / learning
Intel Thread Director [25]	hybrid client CPU	P versus E cores; hardware tracks instruction mix per thread and informs the OS	hardware classification
Windows QoS [27]	OS scheduler	QoS levels may restrict threads to a subset of processors	heuristics and developer hints
Linux EAS [26]	kernel	asymmetric CPU capacity; energy model	none; energy-model lookup
Kubernetes [28]	cluster	extended resources through filter and score plugins	none stated
AWS Batch [29]	cloud batch	EC2, Fargate, ECS and EKS compute	none stated
Slurm [30]	HPC batch	GPUs, MPS and MIG devices as generic resources	none stated
Ray [31]	distributed Python	declared CPU and GPU resources; placement groups	none stated
StarPU [3]	task runtime	CPU and GPU tasks, data transfers	performance models

The products split the work: operating-system features place threads, cluster products place containers or jobs against declared requests, and only workflow runtimes such as StarPU understand task graphs. No product page we read combines application-level DAG knowledge, learned per-resource runtime estimates and published decision-overhead measurements on a laptop-class hybrid CPU/GPU system.

TABLE III

THIS WORK COMPARED WITH THE CLOSEST PUBLISHED SYSTEMS

System	Evaluation	Hardware	Overhead reported	Intervals	Informed baseline
[4]	simulation	simulated cluster	inference time	no	no
[5]	trace simulation + testbed	3 Jetson boards	no	no	no
[16]	real	Zynq CPU+FPGA	yes	no	no
[17]	simulation	simulated heterogeneous cluster	prediction error	median errors	HEFT with true runtimes
[3]	real	CPU+GPU nodes	history models	not assessed	no
[19]	real, small DAGs	CPU/GPU	runtime monitor	not stated	HEFT, StarPU
This work	real executions	P/E cores, RTX 3050	end-to-end and per-dispatch	paired bootstrap, Holm	yes (hand rule, static plans)

Gap. The literature leaves open a like-for-like comparison of static and dynamic scheduling with one cost model, real execution, end-to-end timing and interval statistics, including a baseline that merely knows which tasks are expensive. This paper supplies that comparison for one workload family on one machine.

III. SYSTEM AND METHOD

A. Runtime

The coordinator converts a workflow into a DAG, keeps a ready queue and dispatches tasks to worker processes, each pinned to one logical processor, with a reserved core for the coordinator (Fig. 1). Tasks become ready when all parents completed (the in-degree gating of the base patent [20]). The workers are six P-core processes, seven E-core processes and one CUDA worker G, which runs GPU variants of three image stages. Every policy honours per-task eligible pools through one helper and a violation raises an error. Every output is compared with a CPU reference, and the event log records each dispatch with its decision time, so that planned and actual times can be compared.

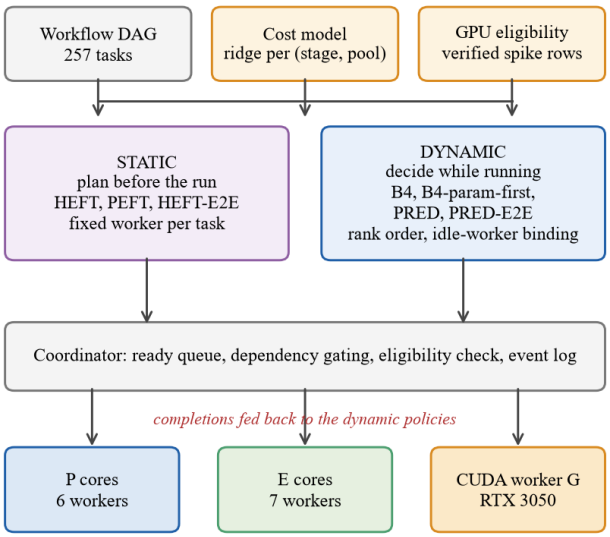


Fig. 1. Framework of the final study: one cost model and one GPU-eligibility table feed both the static planners and the dynamic dispatchers; coordinator, event log and worker pools are shared.

Failed or timed-out tasks get a terminal status and their dependents are marked not-run; every run starts from a clean directory; failed runs are kept and flagged. External-command tasks (the Montage jobs of the earlier study) run as processes without a shell, with timeouts, process-tree kill and hashing of inputs and outputs. A run produces an event log (one JSON line per event) and a manifest; the manifest ties the run to its design, code and environment (Table IV).

TABLE IV

EVENT LOG AND RUN MANIFEST CONTENT

Record	Fields (selection)
task_ready, task_dispatched	task id, worker, pool, queue wait, worker load, decision time, decision record
task_submitted, task_completed	kernel time, start and end timestamps, transfer time, bytes moved, output size and SHA-256
worker_ready, run_started, run_finished	pid, requested and actual affinity, capabilities; makespan, status, tasks per pool
run manifest	condition, policy, seed, instance, warm-up flag, status, output verdict and hashes, event-log SHA-256, git commit and dirty flag, source and design SHA-256
session record	design hash, machine id, environment hash, blocks completed, preflight log, final status

B. Cost model

The model fits, per (stage, pool), a ridge regression on the numeric task parameters and the size that vary in the logs and on their pairwise products; it names no stage or parameter, refuses an unseen stage or pool, and is refitted by one command. An *end-to-end* term adds the artifact set-up time and the inter-process overhead per pool. The CUDA stage time measured by the runtime already contains the host-to-device copy, the kernel and the device-to-host copy (checked in the stage code), so the end-to-end variant never adds them again; for local pools the modelled transfer is zero. The model and the end-to-end terms were fitted on calibration sessions and checked on hold-out sessions that were never used for fitting (Table V).

TABLE V

COST-MODEL ERROR ON HOLD-OUT TASKS (NEVER USED FOR FITTING); E = EFFICIENCY CORE, P = PERFORMANCE CORE, G = CUDA WORKER

Stage on pool	n	MAE (ms)	Median abs. error (%)
aggregate on E	11	0.02	7.2
aggregate on P	13	0.01	7.6
denoise on E	641	19.89	4.0
denoise on G	51	100.40	22.8
denoise on P	844	17.73	10.9
edges on E	254	1.94	4.6
edges on G	607	4.23	30.4
edges on P	675	2.17	11.1
features on E	224	0.44	4.9
features on G	417	2.79	31.3
features on P	895	0.53	10.3
source on E	661	33.39	8.9
source on P	875	20.36	8.8

CPU stages are predicted within 4% to 11% median error, GPU stages within 23% to 31%. The GPU estimate is the weakest part of the model, and it limits what model-based GPU placement can do.

Features and fitting protocol. Features are decision-time safe: task parameters are known when the task becomes ready. Offline features are reconstructed from the logged decision context and a parity test enforces equality with the online features, so that no feature uses information from the future. Inference is a dot product of a few numbers. Over the project more elaborate predictors were tried (Table VI); in the earlier phases none of them improved scheduling over the ridge model enough to justify its cost, so the final study uses the simple model.

TABLE VI

ESTIMATORS TRIED DURING THE PROJECT (ROLE IN THE FINAL STUDY)

Estimator	Description	Role
E0	constant time per task type and pool	earlier phases
E1	solo-profile table keyed by stage and size	earlier phases
E2	analytic size model	earlier phases
E3	moving history of observed times	earlier phases
E1+rc	E1 recalibrated online from completed tasks	earlier phases
E4 ridge	ridge on task parameters and occupancy context	earlier phases
CatBoost, MLP	gradient-boosted trees and a small neural network on the same features	evaluated, not adopted
ParamModel	ridge per (stage, pool) on varying parameters, size and pairwise products	**used in the final study**
E5 oracle	offline replay of observed times	reference only

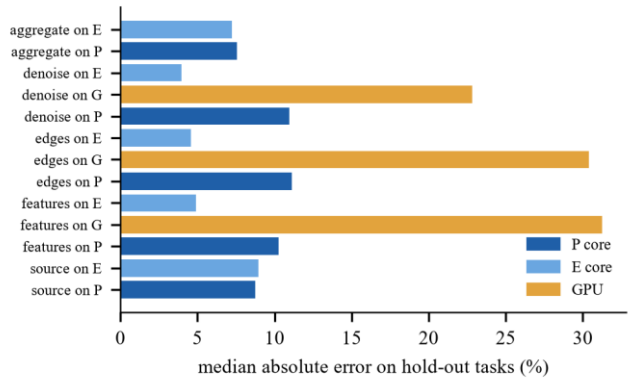


Fig. 2. Median absolute error of the cost model on hold-out tasks, by stage and pool.

C. Static and dynamic policies

TABLE VII

POLICIES COMPARED (ALL USE THE SAME ELIGIBILITY RULES AND, WHERE THEY PREDICT, THE SAME COST MODEL)

Policy	Kind	Rule
B4	dynamic	first idle eligible worker for each ready task in FIFO order, priority P then E; no estimate (the mapping of the base patent's dispatch idea [20])
B4_param_first	dynamic	B4 with the ready queue sorted by the cost-relevant task parameter (a hand rule: human knowledge)
PRED_RANK_GREEDY (PRED)	dynamic	order ready tasks by predicted upward rank; bind only to an *idle* eligible worker with the least predicted kernel time
PRED_RANK_GREEDY_E2E (PRED-E2E)	dynamic	as PRED, choosing the idle worker with the least predicted occupancy = transfer + kernel + set-up + inter-process time
HEFT_param_model	static	HEFT plan from the model, computed before the run; fixed worker per task
PEFT_param_model	static	PEFT plan (optimistic cost table) from the same model
HEFT_E2E	static	HEFT plan with the end-to-end terms added to the costs
HEFT_oracle	static (reference)	HEFT plan from class-median costs of the calibration sessions; reference only, not a bound

The proposed dispatcher orders ready tasks by predicted upward rank, $\text{rank}(t) = \hat{y}(t) + \max_{*c* \in \text{succ}(*t*)} \text{rank}(c)$, and at every dispatch pass proceeds as follows: (1) sort the ready tasks by decreasing rank; (2) for each task consider only idle eligible workers, and leave the task queued if there is none (late binding, work conserving); (3) send the task to the idle worker with the least predicted time; (4) on completion log the observed time and release the dependents. A static plan instead fixes every assignment in advance; its planned and actual makespan are both logged. Policy set-up time (building the plan or the model tables) is recorded separately and is part of the end-to-end makespan.

Static planning. HEFT [1] ranks tasks by upward rank and places each, in rank order, on the processor with the earliest finish time, inserting it into idle gaps:

$$\text{rank}_u(i) = w_i + \max_{*j* \in \text{succ}(*i*)} (c_{ij} + \text{rank}_u(j)) \quad (2)$$

PEFT [2] replaces the rank by the mean of an optimistic cost table and chooses the processor that minimises the finish time plus that table:

$$\text{OCT}(i, k) = \max_{*j* \in \text{succ}(*i*)} \min_{*q*} [\text{OCT}(j, q) + t(j, q) + c_{ij}] \quad (3)$$

Both were reimplemented as static plans from a cost matrix filled by the cost model, and the repository tests reproduce the published worked examples (the 10-task example of the HEFT paper; the PEFT sample as printed in the author's thesis chapter, where PEFT yields a schedule length of 122 and HEFT 133; the only tolerated difference is the tie-break of two equal ranks). The plans are occupancy-blind, as registered in the protocol: they do not model contention for a worker beyond the plan itself, and are not re-planned during the run.

D. GPU eligibility from measurements

A stage may run on the CUDA worker only if a recorded spike measurement shows a verified speed-up at that size and iteration count; a dropped row overrides a pass. Table VIII lists the rows. Three configurations result: *cpu* (P and E cores only), *gpu* (strict: only stages verified on the original record) and *ext* (strict plus denoise at the iteration counts verified on the extended record). No configuration is GPU-only: the GPU is one narrow resource, and one CUDA worker was used throughout.

TABLE VIII

GPU SPIKE RECORDS: TIME PER TASK ON THE CPU AND ON THE GPU (INCLUDING HOST-DEVICE COPIES) AND THE VERDICT

Record	Stage (denoise iterations)	CPU ms	GPU ms	Speed-up	Verdict
original	denoise (1)	6.4	6.7	0.96×	dropped
original	denoise (8)	48.8	41.9	1.16×	pass
original	edges	5.8	0.8	7.68×	pass
original	features	2.3	0.8	2.88×	pass
extended	denoise (2)	11.5	11.6	0.99×	dropped
extended	denoise (6)	33.9	31.9	1.06×	pass
extended	denoise (15)	92.5	77.5	1.19×	pass
extended	denoise (40)	359.4	203.3	1.77×	pass
extended	denoise (120)	721.6	603.6	1.20×	pass

E. End-to-end timing convention

The end-to-end makespan runs from the start of `run()` to the last completion; initialisation and policy set-up are recorded separately (amendment F1 of the protocol). The older convention, first dispatch to last completion, hides the time a static plan needs to be computed and is kept for comparison. Adding the two conventions to the analysis changed no conclusion on this workload size (Table XVIII).

F. Design decisions

- **One cost model for both sides.** Static and dynamic policies use the same ridge model, so a difference between them is

a difference in *when and how the estimates are used*, not in how good the estimates are.

- **End-to-end makespan as the primary metric.** A static plan has a planning cost that the first-dispatch convention hides; the end-to-end metric charges it (12.5 ms to 81.3 ms here).
- **A strict GPU gate.** A stage is allowed on the GPU only where a measured speed-up exists, which keeps the comparison from being decided by a bad GPU placement rule; the extended set adds only iteration counts verified on a second record.
- **A parameter-aware baseline.** B4_param_first is the simplest policy that knows what the model knows; it tests whether learning is needed at all.
- **An oracle only as a reference.** HEFT_oracle uses class-median costs and is not a bound; no policy is compared against hindsight.
- **A decision rule for the hybrid.** A B4-or-PRED selector would be built only if its headroom exceeded the ±2% margin in the development cells; it did not, so none was built and no held-out hybrid session was run.

IV. Experimental Setup

Machine. Intel Core i5-13500HX (6 P, 8 E cores), NVIDIA RTX 3050 6 GB (driver 581.83, CUDA 12.8 build of PyTorch 2.11), 15.7 GB RAM, native Windows 11; sessions record an environment hash and are never pooled across machines. **Workloads.** The image pipeline (source, denoise, edges, features, aggregate; 512 px, 64 chains, 257 tasks) in which *H* chains are heavy (120 denoise iterations) and the rest light (2); the heavy parameter is invisible to a size-keyed table (Table IX). The *primary* workload h16_l120 has 16 heavy chains. Others vary the heavy count (4, 40), the cost ratio (heavy 40 or 15 iterations: ratio 20 and 7.5) and include a uniform workload with no spread in cost.

TABLE IX

Evaluation workloads (runs per session, included runs in brackets; the first block of each session is a warm-up)

Workload	Heavy chains / iterations	Blocks	Runs
h16_l120 (primary)	16, 120 vs 2 (ratio 60)	25	600 (576)
uniform	none; cpu and gpu only	13	208 (192)
h04_l120	4, 120 vs 2	13	312 (288)
h40_l120	40, 120 vs 2	13	312 (288)
h16_l40	16, 40 vs 2 (ratio 20)	13	312 (288)
h16_l15	16, 15 vs 2 (ratio 7.5)	13	312 (288)

Measured heterogeneity. A 512-pixel denoise task with 30 iterations takes 156 ms on a P core and 560 ms on an E core (3.6×); the two shorter stages are 2.8× to 3.3× slower on an E core. On the GPU the edge-detection stage is up to 15× faster at 1024 px but only breaks even at small tiles because host-device copies dominate (Fig. 3). A scheduler that ignores these ratios sends heavy work to slow cores.

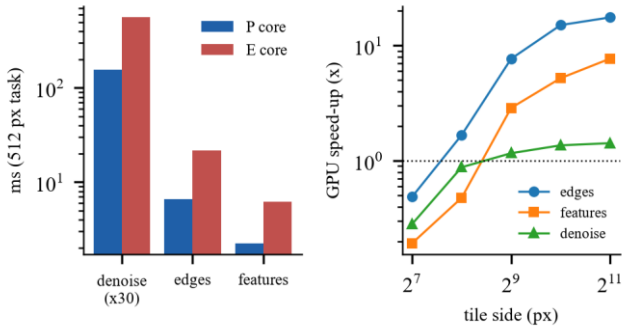


Fig. 3. Measured heterogeneity: stage kernel time on a P and an E core (left); GPU speed-up versus tile size (right).

Earlier workloads. The earlier study also used the 59-task Montage mosaic [32], [33] (native Windows build, nine DSS2 images, three identical sequential references of 1459 s to 1469 s with identical output hashes) and a two-factor batch; Table X lists the reference runs.

TABLE X

Montage reference runs: median wall time per task type (three identical sequential runs of 59 tasks)

Task type	n	Median (s)	Range (s)
mProject (reprojection)	27	160.57	141.02–169.89
mDiffFit (overlap fit)	108	0.251	0.041–1.036
mBackground	27	0.336	0.303–0.610
mConcatFit / mBgModel / mImgtbl	3 each	0.207 / 0.379 / 0.094	–
mAdd / mViewer	3 each	0.870 / 4.157	–
Total sequential makespan	3 runs	1459.4, 1467.1, 1469.3	outputs identical

Data separation. Calibration sessions (cal_l15, cal_l40, cal_l120, cal_uniform: 99 runs in their completed retries) fitted the model and the end-to-end terms; hold-out sessions (hold_l40, hold_l120: 28 runs) checked them; the evaluation sessions came afterwards with new chain orders. The protocol and the hashes of the evaluation designs were logged as frozen before the first evaluation session. **Protocol.** Conditions run once per block in seeded random order with one worker pool for all conditions. Before every block free memory had to exceed 2.4 GB at the start of a session and 0.8 GB per block, and the laptop had to be on mains power; these thresholds were never lowered, and attempts that failed the gate were kept on disk (suffix _r2 ... _r8) and never pooled. Output correctness was verified for every included run.

Statistics. The effect of a condition *c* against a reference *r* is computed per block *b* from the makespans *M*, with a percentile bootstrap interval (10,000 resamples) stratified by chain order and a sign-flip *p*-value:

$$\text{effect} = (\exp(\ \text{mean}_{*b*}\ \ln(\ M_{*c*,*b*} / M_{*r*,*b*}\ )\ ) - 1) \times 100\% \quad (1)$$

Negative values mean that *c* is faster. The confirmatory family is the ten contrasts C1–C10 on the primary session under Holm correction with a ±2% equivalence margin; everything else, including the other five workloads and the comparison with the hand rule, is exploratory and unadjusted. For each contrast the expected outcome was written before the data (stated in the protocol) so that wrong predictions can be reported: the expectations held for C1, C2, C7, C9 and C10, and were wrong for C3 and C4 (we expected an inconclusive or equivalent result; the dynamic policy was about 5% faster).

TABLE XI

EXPECTATIONS WRITTEN IN THE PROTOCOL BEFORE THE EVALUATION DATA, AND WHAT THE DATA SHOWED

Id	Expected before the data	Outcome	Held?
C1	faster (earlier sessions: about 19%)	faster, −17.8%	yes
C2	equivalent	equivalent, +0.6%	yes
C3	inconclusive or equivalent	faster, −5.2%	**no**
C4	inconclusive or equivalent	faster, −5.2%	**no**
C5	none stated	faster, −12.2%	–
C6	none stated	faster, −8.5%	–
C7	faster or equivalent	equivalent, −0.0%	yes
C8	none stated	faster, −21.2%	–
C9	small gain	faster, −2.6%	yes
C10	small	inconclusive, +1.6%	yes

V. RESULTS

A. Primary workload

Table XII gives the end-to-end makespan of every policy in the three hardware sets and Fig. 4 adds the 95% intervals. In every set, the three cost-aware dynamic policies (B4 param-first, PRED, PRED-E2E) are within about 1% of each other and clearly faster than blind B4; the static HEFT plan is slower than PRED, and PEFT is the slowest static plan. All 576 included runs of the session had verified output.

TABLE XII

PRIMARY WORKLOAD H16_L120: END-TO-END MAKESPAN IN SECONDS (GEOMETRIC MEAN OVER 24 INCLUDED BLOCKS)

Policy	CPU only	Strict GPU	Extended GPU
B4 (blind, dynamic)	4.010	4.075	4.123
B4 param-first (rule, dynamic)	3.310	3.218	3.069
PRED (dynamic)	3.297	3.213	3.050
PRED-E2E (dynamic)	3.318	3.212	3.033
HEFT (static)	3.478	3.660	3.114
HEFT-E2E (static)	3.499	3.512	3.173
PEFT (static)	4.093	4.050	3.689

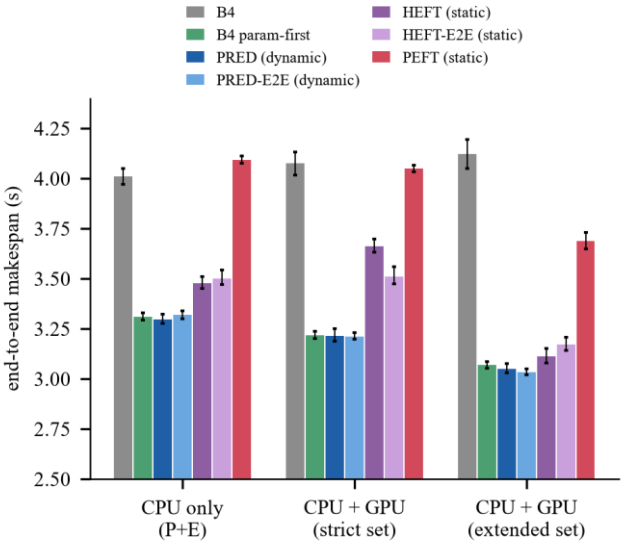


Fig. 4. End-to-end makespan of each policy in the three hardware sets on the primary workload (geometric mean, 95% interval, 24 blocks).

B. Confirmatory contrasts

Table XIII and Fig. 5 give the ten pre-stated contrasts. The dynamic dispatcher beats blind B4 by 17.8% on the CPU (C1) and 21.2% with the strict GPU set (C8). Against the static HEFT plan built from the *same* model it is faster by 5.2% on the CPU (C3) and by 12.2% with the strict GPU set (C5); the end-to-end variants give 5.2% (C4) and 8.5% (C6). Adding the end-to-end cost terms to the dynamic policy changes nothing (C2, C7: equivalent within ±2%). Using the strict GPU set helps the dispatcher by 2.6% (C9) but not blind B4 (C10, +1.6%, inconclusive).

TABLE XIII

CONFIRMATORY CONTRASTS (HOLM FAMILY OF TEN; EFFECT ON END-TO-END MAKESPAN, NEGATIVE = FIRST POLICY FASTER)

Id	Contrast	Effect % [95% CI]	Holm p	Result
C1	PRED vs B4 (CPU)	−17.8 [−18.8, −16.7]	0.001	faster
C2	PRED-E2E vs PRED (CPU)	+0.6 [+0.1, +1.3]	0.141	equivalent (within ±2%)
C3	PRED vs HEFT (CPU)	−5.2 [−5.9, −4.6]	0.001	faster
C4	PRED-E2E vs HEFT-E2E (CPU)	−5.2 [−5.9, −4.5]	0.001	faster
C5	PRED vs HEFT (strict GPU)	−12.2 [−13.4, −11.1]	0.001	faster
C6	PRED-E2E vs HEFT-E2E (strict GPU)	−8.5 [−9.7, −7.6]	0.001	faster
C7	PRED-E2E vs PRED (strict GPU)	0.0 [−0.7, +0.5]	0.960	equivalent (within ±2%)
C8	PRED vs B4 (strict GPU)	−21.2 [−22.6, −19.7]	0.001	faster
C9	PRED, strict GPU vs CPU	−2.6 [−3.3, −1.7]	0.001	faster
C10	B4, strict GPU vs CPU	+1.6 [+0.2, +3.0]	0.141	inconclusive

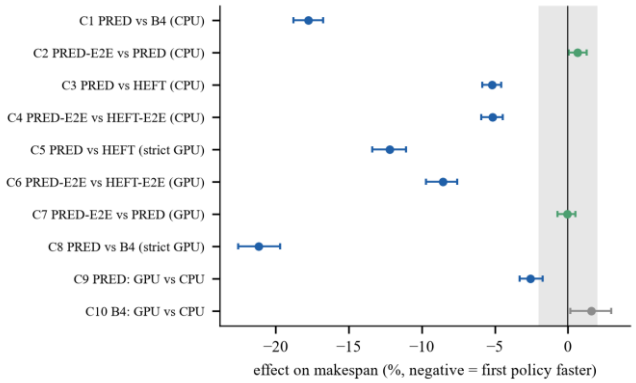


Fig. 5. Confirmatory contrasts C1–C10 with 95% intervals; the grey band is the ±2% equivalence margin.

C. Six workloads

The advantage depends on how much task cost varies (Fig. 6; Table XIV and Table XV, exploratory outside the primary session). Against the static HEFT plan the dynamic dispatcher gains 0.2% to 5.2% on the CPU and 3.7% to 17.1% with the strict GPU set. Against blind B4 it gains almost 40% when only four of 64 chains are heavy, 2.8% to 4.8% on the CPU (6.0% to 7.9% with the strict set) when 40 are heavy or the cost ratio is 20, and nothing on the uniform workload; with the lowest cost ratio (7.5) it is slightly *slower* than B4 with the strict GPU set (+1.1% [+0.3, +1.9]). With little spread in task cost there is little to order, and the dispatcher's own cost (about 9 ms more scheduling time than B4) is no longer paid back.

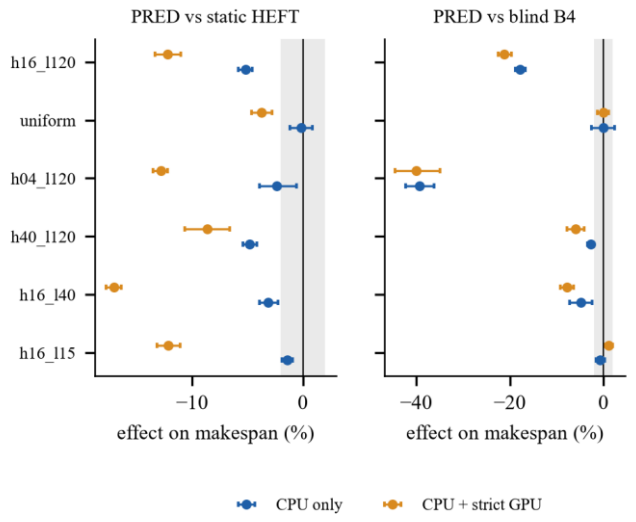


Fig. 6. PRED against static HEFT (left) and blind B4 (right) on the six workloads, CPU only (blue) and with the strict GPU set (orange); exploratory except the first row.

TABLE XIV

CPU ONLY: EFFECT OF PRED ON END-TO-END MAKESPAN AGAINST STATIC HEFT AND AGAINST BLIND B4 (%, NEGATIVE = PRED FASTER; EXPLORATORY EXCEPT H16_L120)

Workload	vs HEFT [95% CI]	vs B4 [95% CI]
h16_l120	−5.2 [−5.9, −4.6]	−17.8 [−18.8, −16.7]
uniform	−0.2 [−1.2, +0.9]	+0.0 [−2.5, +2.3]
h04_l120	−2.4 [−3.9, −0.6]	−39.4 [−42.3, −36.3]
h40_l120	−4.8 [−5.4, −4.2]	−2.8 [−3.4, −2.2]
h16_l40	−3.2 [−3.9, −2.3]	−4.8 [−7.3, −2.4]
h16_l15	−1.4 [−1.9, −0.9]	−0.7 [−1.5, +0.2]

TABLE XV

STRICT GPU SET: EFFECT OF PRED ON END-TO-END MAKESPAN AGAINST STATIC HEFT AND AGAINST BLIND B4 (%, EXPLORATORY EXCEPT H16_L120)

Workload	vs HEFT [95% CI]	vs B4 [95% CI]
h16_l120	−12.2 [−13.3, −11.1]	−21.2 [−22.5, −19.7]
uniform	−3.7 [−4.7, −2.8]	0.0 [−1.3, +1.1]
h04_l120	−12.8 [−13.6, −12.2]	−40.1 [−44.5, −35.0]
h40_l120	−8.6 [−10.7, −6.6]	−6.0 [−7.8, −4.2]
h16_l40	−17.1 [−17.8, −16.4]	−7.9 [−9.2, −6.4]
h16_l15	−12.2 [−13.2, −11.1]	+1.1 [+0.3, +1.9]

D. GPU use

Table XVI shows the effect of the hardware set within each policy. The GPU helps the model-driven dispatcher (strict set 2.6%, extended set 7.5% faster than the CPU only) and the static HEFT plan on the extended set (10.5%), but not blind B4, which sends work to the GPU without a cost model (32% of its tasks with the strict set) and is slower with it. Under the model-based policies the CUDA worker takes roughly 15% to 20% of the 257 tasks (Fig. 7); its busy fraction is higher with the extended set. One CUDA worker is a single narrow resource, so the benefit comes from putting the right tasks on it, which needs the cost model.

TABLE XVI

EFFECT OF THE HARDWARE SET WITHIN EACH POLICY ON THE PRIMARY WORKLOAD (%, NEGATIVE = FASTER)

Policy	Strict vs CPU	Extended vs CPU	Extended vs strict
PRED (dynamic)	−2.6 [−3.3, −1.7]	−7.5 [−7.8, −7.2]	−5.1 [−5.7, −4.4]
B4 (blind)	+1.6 [+0.2, +3.0]	+2.8 [+0.8, +4.9]	+1.2 [−0.5, +3.1]
HEFT (static)	+5.2 [+4.2, +6.6]	−10.5 [−11.5, −9.5]	−14.9 [−16.2, −13.7]

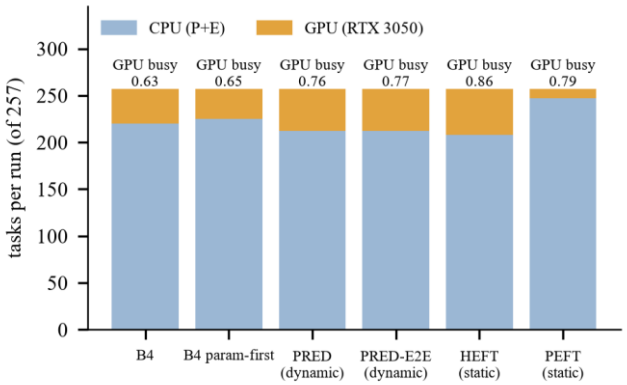


Fig. 7. Tasks run on the CPU and on the GPU per policy (extended set, primary workload) and the GPU busy fraction.

E. End-to-end terms, timing convention and overhead

Adding set-up and inter-process terms did not improve the dynamic policy (C2: +0.6% [+0.1, +1.3]; C7: −0.0% [−0.7, +0.5]) and on the uniform workload with the strict set the end-to-end variant was +2.9% [+0.4, +5.9] slower than B4: the terms are not free of cost when the extra decision work has nothing to gain. Adding them to the static plan did not help consistently (Table XVII). PEFT is clearly the slowest static plan. The oracle HEFT plan, built from class-median costs of the calibration sessions, is a reference, not a bound: with realised per-run times a static plan could be better.

TABLE XVII

STATIC PLANS AGAINST HEFT_PARAM_MODEL ON THE PRIMARY WORKLOAD (%, EXPLORATORY)

Hardware set	HEFT_E2E	PEFT	HEFT oracle (reference)
CPU only	+0.6 [−0.3, +1.6]	+17.7 [+16.8, +18.6]	−0.2 [−1.0, +0.5]
strict GPU	−4.1 [−5.5, −2.5]	+10.6 [+9.5, +11.5]	−3.1 [−4.3, −2.0]
extended GPU	+1.9 [+0.5, +3.3]	+18.5 [+16.9, +20.1]	+3.3 [+1.4, +5.5]

The two timing conventions differ by the initialisation plus the policy set-up (Table XVIII). The difference is 12.5 ms for B4 and 81.3 ms for PEFT on a makespan of about 3 to 4 s, so no conclusion flips; on the old convention PRED against HEFT on the CPU is −4.9% [−5.6, −4.3] against −5.2% on the end-to-end convention. Policy set-up costs 19 ms (PRED), 30 ms (HEFT) and 71 ms (PEFT); total scheduling time is 16, 29 and 31 ms.

TABLE XVIII

PRIMARY WORKLOAD, CPU ONLY: MAKESPAN UNDER THE OLD CONVENTION (FIRST DISPATCH TO LAST COMPLETION) AND THE END-TO-END CONVENTION (S) AND THE DIFFERENCE (MS)

Policy	Old	End-to-end	Diff. (ms)
B4 (blind, dynamic)	3.998	4.010	12.5
B4 param-first (rule, dynamic)	3.297	3.310	12.8
PRED (dynamic)	3.268	3.297	29.2
PRED-E2E (dynamic)	3.288	3.318	30.8
HEFT (static)	3.437	3.478	40.7
HEFT-E2E (static)	3.457	3.499	42.4
PEFT (static)	4.012	4.093	81.3

TABLE XIX

PRIMARY WORKLOAD, CPU ONLY: INITIALISATION, POLICY SET-UP AND TOTAL SCHEDULING TIME PER RUN (MS) AND MEAN QUEUE WAIT (S)

Policy	Init.	Policy set-up	Scheduling	Queue wait
B4 (blind, dynamic)	7.7	2.6	6.7	0.571
B4 param-first (rule, dynamic)	8.0	2.6	6.6	0.560
PRED (dynamic)	7.7	19.0	16.0	0.624
PRED-E2E (dynamic)	7.8	20.5	22.8	0.626
HEFT (static)	7.6	30.4	29.1	0.613
HEFT-E2E (static)	7.6	32.2	29.4	0.615
PEFT (static)	7.6	71.1	31.0	0.554

F. Workload by workload

Uniform (no spread in cost). All policies are within a few percent: on the CPU B4 2.30 s, B4 param-first 2.26 s, PRED 2.30 s, HEFT 2.31 s; there is nothing to order and the dispatcher ties blind dispatch (+0.0% [−2.5, +2.3]). **Four heavy chains (h04_l120).** The heavy chains decide the critical path; blind B4 starts them late or on a slow core and takes 3.84 s against 2.33 s for PRED (−39.4%); the static HEFT plan is also close (2.39 s). **Forty heavy chains (h40_l120).** Most work is heavy, so order matters less: B4 6.15 s, PRED 5.98 s, HEFT 6.28 s. **Lower cost ratios (h16_l40, h16_l15).** With ratios of 20 and 7.5 the gaps shrink: on the CPU PRED is 2.36 s against B4 2.48 s at ratio 20 and 2.03 s against 2.04 s at ratio 7.5, while the strict GPU set still separates the static plan from the dynamic policies (HEFT 2.24 s against PRED 1.96 s at ratio 7.5): when the GPU model error matters, committing to the estimate in advance costs more than deciding late.

Precision. With 24 paired blocks the half-width of the ten confirmatory intervals is 0.6 to 1.4 percentage points; the exploratory sessions use 12 blocks and are less precise, so they show direction and order of magnitude and are not used to rank near-equal policies. The 10 or 24 blocks of a session repeat a few chain orders, so intervals describe run-to-run noise, not variation across workloads, which is why six workloads and, earlier, twenty unseen chain orders were used.

Planned versus actual. The static plans are not far from reality in total (Table XX), yet a fixed assignment cannot react

to the actual completion order. A ratio below 1 means that the plan was optimistic.

TABLE XX

PLANNED OVER ACTUAL MAKESPAN OF THE STATIC PLANS (PRIMARY WORKLOAD)

Plan	CPU only	Strict GPU	Extended GPU
HEFT	0.94	0.88	0.90
PEFT	0.94	0.95	0.95
HEFT-E2E	1.03	0.99	0.97
HEFT oracle (reference)	0.94	0.91	0.83

G. A simple rule and a hybrid selector

Against B4 param-first, the queue sorted by the cost parameter, PRED is −0.4% [−0.9, +0.1] on the CPU, −0.6% [−1.1, −0.0] with the extended set and −0.2% [−0.9, +0.7] with the strict set (exploratory): under 1% in every configuration. Most of the improvement over blind B4 therefore comes from using the task parameter to choose the order and the place, not from rank ordering and late binding as such. A hybrid selector between B4 and PRED would be worth building only if its headroom exceeded the ±2% margin; in 17 development cells the maximum headroom was 1.1%, the mean 0.08%, and no cell exceeded the margin, so none was built.

H. Earlier exploratory study

An earlier study used the same runtime on a different cost-skewed family. With 16 heavy chains among 64, the dispatcher shortened makespan by 19.3% (6P+7E, 12 chain orders never used before) and 20.5% (2P+7E, 8 orders) against B4, and was level with the hand rule (+0.66%, +0.27%); the gain was about 27% with few heavy chains and about 4% when most were heavy (Table XXI, Fig. 8). There was no gain on the 59-task Montage mosaic (B4 was 47.9% slower than static P-only, because every task prefers the P cores by 2.3× to 3.6×), on a uniform batch (+0.45%) or on a two-factor batch. In that session HEFT with the same model was level with the dispatcher (+1.10% [−0.53, +2.81]) and PEFT was not; the final session finds the dispatcher about 5% faster than HEFT on the CPU. The timing convention does not explain the difference (Table XVIII); the sessions differ in chain orders, block count and code state, and the difference was not isolated, so both results are reported.

TABLE XXI

EARLIER EXPLORATORY SESSIONS ON UNSEEN CHAIN ORDERS (EFFECT AGAINST B4, NEGATIVE = SHORTER)

Session	Blocks / orders	Dispatcher	Hand rule
6P+7E, 16 heavy	36 / 12	−19.32% [−20.0, −18.6]	−19.85%
2P+7E, 16 heavy	24 / 8	−20.45% [−21.2, −19.8]	−20.67%
6P+7E, 4 heavy	12 / 4	−27.65% [−34.0, −18.7]	−26.94%
6P+7E, 8 heavy	12 / 4	−26.76% [−27.9, −25.7]	−26.93%
6P+7E, 40 heavy	12 / 4	−3.81% [−5.1, −1.9]	−5.15%

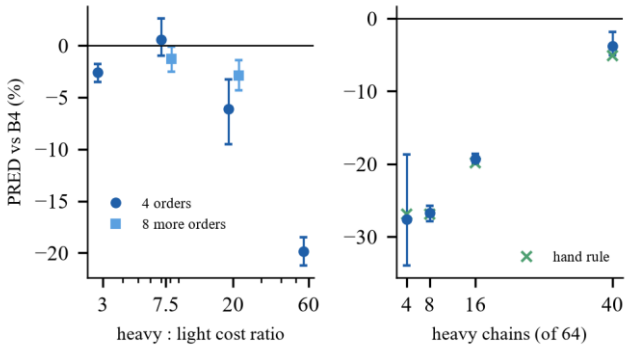


Fig. 8. Earlier study: gain over B4 versus heavy:light cost ratio (left) and number of heavy chains among 64 (right).

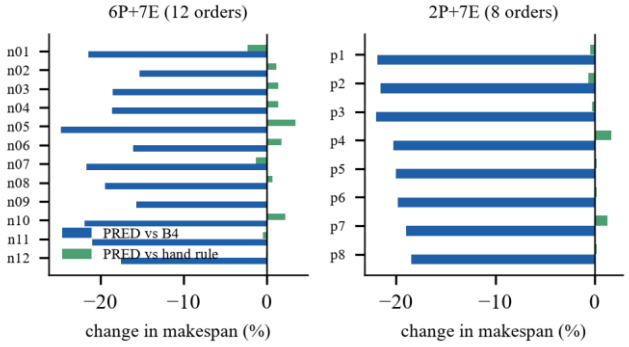


Fig. 9. Earlier study: change in makespan of the dispatcher against B4 (blue) and the hand rule (green) for each chain order not used before (12 orders left, 8 orders right).

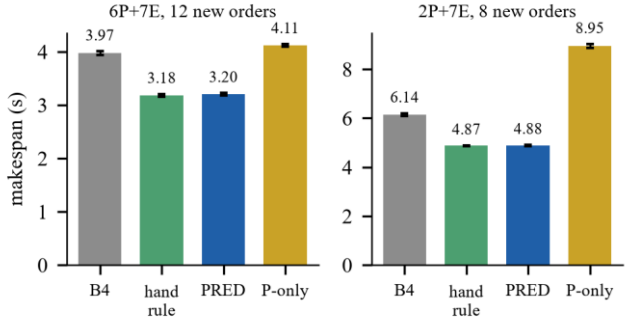


Fig. 10. Earlier study: mean makespan by policy on unseen chain orders (geometric means with 95% intervals): 12 orders on 6P+7E (left), 8 orders on 2P+7E (right).

Overhead and a defect found while measuring it. Two overhead figures exist: the per-dispatch decision time (6.8 ms per run for B4 and 15.5 ms for the dispatcher, about 0.5% of makespan) and the coordinator scheduling time (0.197 s and 0.100 s). The difference was traced with an instrumented run: an idle CUDA worker that no task can use defeats the early exit of the dispatch loop, and 10,042 of 10,299 selection calls of B4 returned nothing. A decision-preserving fix in a separate adapter cut the coordinator time to 8 ms and 24 ms and moved the dispatcher's gain from −19.47% to −20.28%, i.e. 0.8 points (78 runs verified; the fix is not applied to the source).

Overhead therefore costs 2% to 5% of these 3 s to 9 s runs but does not explain the result. The final benchmark keeps the idle CUDA worker in the pool in the *cpu* configuration, which is one candidate for the unresolved difference between the two sessions.

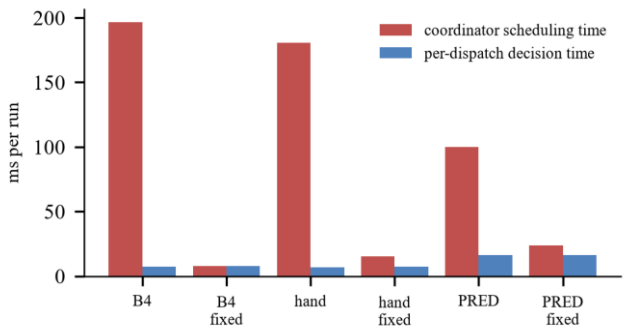


Fig. 11. Earlier study: per-dispatch decision time versus coordinator scheduling time, before and after the dispatch-loop fix.

Remote worker (emulated, not rerun). In earlier registered sessions on an emulated link (a node on one local E core behind a loopback socket and a delay shim), blind remote placement on a slow link cost 31% and transfer-aware placement avoided that harm; on a fast link the remote worker gave no benefit although the predicted execution and transfer times matched the measured ones (0.1457 s against 0.1464 s, and 0.0937 s against 0.0943 s; the predictive policy was 3.78% [+2.27, +5.58] slower with the remote worker). These are emulation results and were not repeated with the final policies; a real remote benefit remains unproven, and no Raspberry Pi or real-network measurement exists.

VI. Discussion

What dynamic dispatch buys. On the CPU the dynamic dispatcher beats the static plan by about 5%, with the strict GPU set by 8% to 12%, and with the extended set by 2% to 3%. The static plan is not wrong in total (planned over actual makespan was between 0.88 and 1.03), but the fixed assignment cannot react to the actual completion order, and its GPU assignments inherit the 23% to 31% model error in advance; a dynamic policy uses the estimate only when a worker is idle. Part of the gap on the GPU sets may therefore reflect model error and not only the timing of decisions; this was not separated experimentally. The dominant effect against blind dispatch is ordering and informed placement: serving predicted-heavy work first removes the tail that blind dispatch creates and keeps heavy chains off the slow cores.

What it does not buy. Not beyond a rule that knows the cost parameter (within 1%); not where task cost does not vary (uniform, and the light workloads); not through end-to-end cost terms; not through a hybrid selector; and not on the GPU for blind dispatch. The dispatcher's value is that it replaces knowledge a person would hard-code and is refitted from logs for a new pool layout. This agrees with the observation of [18] that order can matter more than estimate accuracy, now on real executions, and with [17] that cheap models suffice for list scheduling.

Ordering versus binding. Two design choices are bundled in the dispatcher and the earlier data separate them. Ordering by predicted rank with early binding to the earliest-finish worker gave −13.21% against B4, whereas ordering plus idle-only (late) binding gave −18.97% on the same six orders, so late binding contributes about six of the nineteen points; the static HEFT plan reached −18.08% by inserting tasks into idle gaps. A plan is brittle when durations are mispredicted and a dispatcher is not; the final benchmark supports this on the GPU sets, where the model error is largest, but does not test mispredicted durations in isolation.

Relation to the base patent and to learned ranking. The base patent [20] gates dispatch on eligibility and dependency counters and recites no prediction; our B4 is a comparison baseline mapped from it, not an implementation of the patent. Learned ranking of DAG tasks on heterogeneous hardware is disclosed in [21]; the contribution here is therefore not the idea of ranking but a measured comparison of static and dynamic scheduling with one cost model, end-to-end timing and an informed baseline. This framing is deliberate: it is what the evidence supports.

Practical reading. For an engineer choosing a scheduler for a small hybrid machine the measurements suggest three steps: first check whether tasks differ in cost by a factor that size alone does not reveal; if they do, order the queue by (predicted) cost before worrying about placement, since a rule that sorts by the right parameter was as good as the learned model in every configuration; and send only stages with a measured GPU speed-up to the GPU. A learned model is the right tool when the parameter is not known in advance or changes with the machine, since it is refitted from logs. **Baselines.** A fair evaluation of a learned scheduler needs an informed rule, static plans from the same model, intervals and the scheduler's own cost; each of these changed a conclusion in this project.

TABLE XXII

GUIDANCE DERIVED FROM THE MEASUREMENTS (THIS WORKLOAD FAMILY AND MACHINE)

Situation	Expected benefit of dynamic, model-driven dispatch
Task cost varies and a few heavy tasks compete for fast workers	large against blind dispatch (17% to 40%); 2% to 12% against a static plan; a rule that knows the parameter gets almost the same
Little or no spread in task cost	none; equal to blind dispatch within ±2%; a small loss is possible with the strict GPU set
A GPU is available for some stages	worth 3% to 8% for the dispatcher if the GPU is used only where a speed-up was measured; harmful for blind dispatch
Model error is large (GPU: 23% to 31%)	prefer dynamic decisions, which use the estimate only when a worker is idle; improve the model before adding complexity
A remote worker behind a link	none in the emulated regimes tested; measure on the real link first

Threats to validity. One machine, one workload family (an image pipeline of 257 tasks at 512 px), one CUDA worker; the workload family was designed to contain a hidden cost. Only the primary session is confirmatory; the other five sessions and the comparison with the hand rule are exploratory and unadjusted. HEFT, PEFT and the oracle plan are occupancy-blind static plans without re-planning; a plan with perfect knowledge of realised task times was not computed. The features of the hybrid check were computed after the primary results had been read (the model and the DAG alone determine them). Runs were made with a dirty working tree and several source hashes, so sessions are never pooled. The remote worker was not rerun and learned DRL schedulers were not run.
Disclosure. Software, analysis scripts and parts of the text were produced with the help of an AI coding assistant under the authors' direction; every number was checked against stored result files. [Authors to confirm the wording required by the venue.]

VII. REPRODUCIBILITY AND AVAILABILITY

Every session stores its design, environment and code hashes. The protocol, the amendment, the preservation record (git state, source hashes, backups) and the activity log of the final study are in the repository package; the tables of this paper are generated from the stored analysis files by scripts, and 521 automated tests pass. The pipeline runs as `scripts/final_stage1.ps1` (calibration, models, designs), `scripts/final_stage2.ps1` (six evaluation sessions and analyses), `scripts/final_analyze.py`, `scripts/final_results_table.py` and `scripts/final_build_report.py`. The runs need mains power and at least 2.4 GB of free memory.

A. Implementation and tests

The framework is Python on Windows. Table XXIII lists its size; the 521 automated tests (run before the dashboard variants were added) cover the coordinator contract, the estimators, the static planners (including the published worked examples), the final policies, GPU eligibility, the statistics and the claims registry, which is re-read against the stored tables by a test. A local web dashboard shows the system state, a replay of runs and the stored results; two later versions in separate packages show the GPU with live load and add a tab that runs one live block of the final benchmark.

TABLE XXIII

IMPLEMENTATION INVENTORY (LINES OF PYTHON COUNTED IN THE REPOSITORY)

Part	Files	Lines
Runtime package (src/caosched)	107	12,578
core: coordinator, pools, DAG, tasks	13	2,026
estimators; schedulers; analysis; experiment		1,197; 845; 1,037; 1,126
final study: static baselines, E2E dispatcher, GPU region	4	216
dashboard (original; v2; v3)		774; 202; 266
Scripts	89	9,709
Tests	52	7,263

B. Incidents and defects

Defects and operational incidents are recorded rather than hidden (Table XXIV). None of them changed a reported number: runs that failed a gate were kept on disk and never pooled.

TABLE XXIV

DEFECTS AND INCIDENTS FOUND DURING THE WORK AND WHAT WAS DONE

Item	Found by	Action
Static plans were timed from the first dispatch, hiding their planning time	design review	end-to-end makespan (amendment F1); old metric kept
Stage 1 aborted when the laptop was unplugged, on retry-id collisions and on low free memory (antivirus, Docker, WSL)	run logs	mains wait and first-unused-id logic; the 2.4 GB and 0.8 GB thresholds were never lowered; aborted attempts kept (_r2 … _r8)
Learned model applied to a profile it was not fitted on	audit of earlier phases	profile binding recorded in the model; refusal on mismatch
Worker-pool rebuild between conditions created cold workers	pilot analysis	one pool for all conditions
Idle CUDA worker defeats the dispatch loop's early exit	overhead diagnosis	quantified; fix in a separate adapter; not applied to the source
Remote node served one client then exited	loopback rehearsal	persistent mode and test

C. Status of the claims

TABLE XXV

CLAIM STATUS AFTER THE PROJECT (SELECTION)

Status	Claim
Supported	dynamic PRED beats blind B4 by 17.8% (CPU) and 21.2% (strict GPU); beats a static HEFT plan from the same model by 5.2% and 12.2%; the GPU helps PRED (2.6% strict, 7.5% extended); end-to-end terms are equivalent
Not established	advantage over a rule that knows the cost parameter (within 1%); a hybrid selector; GPU benefit for blind B4; a remote-offloading benefit; Raspberry Pi results; any claim beyond this machine and workload family
Not supported / null	gain on Montage, on the uniform and light workloads and on the two-factor workload; a remote worker helping EFT at compute level 30

VIII. Conclusion

On a hybrid CPU+GPU laptop and one workload family, dynamic dispatch driven by a cheap ridge cost model finishes a workflow 2% to 12% sooner than a static HEFT plan from the same model (5% on the CPU, 12% with the strict GPU set) and about 18% to 21% sooner than blind dispatch, when task cost varies; it ties blind dispatch when it does not. A rule that knows the cost parameter reaches almost the same result, end-to-end cost terms and a hybrid selector add nothing, and the GPU helps only when it is used where a speed-up was measured and the model chooses the tasks. Future work: a GPU cost model with lower error and an experiment that separates model error from decision timing, a second workload family, the remote worker with the final policies and a real link, and a clean-tree rerun of the headline session.

Availability

An anonymised artefact (code, protocols, claims registry and result tables) will be provided to reviewers on request.

Appendix A. End-to-end makespan of all policies and workloads (s)

TABLE XXVI

All workloads, CPU only: end-to-end makespan in seconds (geometric mean; 24 blocks for h16_l120, 12 for the others)

Workload	B4	B4 param-first	PRED	PRED-E2E	HEFT	HEFT-E2E	PEFT
h16_l120	4.010	3.310	3.297	3.318	3.478	3.499	4.093
uniform	2.302	2.262	2.302	2.301	2.306	2.331	2.362
h04_l120	3.840	2.352	2.329	2.322	2.385	2.415	2.752
h40_l120	6.147	6.021	5.976	5.959	6.279	6.426	6.343
h16_l40	2.475	2.382	2.355	2.404	2.432	2.474	2.617
h16_l15	2.041	2.021	2.027	2.038	2.057	2.107	2.129

TABLE XXVII

All workloads, strict GPU set: end-to-end makespan in seconds (geometric mean; 24 blocks for h16_l120, 12 for the others)

Workload	B4	B4 param-first	PRED	PRED-E2E	HEFT	HEFT-E2E	PEFT
h16_l120	4.075	3.218	3.213	3.212	3.660	3.512	4.050
uniform	2.089	2.120	2.088	2.150	2.169	2.192	2.200
h04_l120	3.742	2.210	2.243	2.233	2.573	2.461	2.753
h40_l120	6.228	5.837	5.855	5.845	6.408	6.293	6.505
h16_l40	2.448	2.254	2.255	2.238	2.719	2.526	2.828
h16_l15	1.943	1.939	1.964	1.972	2.237	2.046	2.275

TABLE XXVIII

All workloads, extended GPU set: end-to-end makespan in seconds (geometric mean; 24 blocks for h16_l120, 12 for the others)

Workload	B4	B4 param-first	PRED	PRED-E2E	HEFT	HEFT-E2E	PEFT
h16_l120	4.123	3.069	3.050	3.033	3.114	3.173	3.689
h04_l120	3.063	2.114	2.144	2.136	2.240	2.282	2.688
h40_l120	6.204	5.676	5.645	5.632	5.734	5.783	6.300
h16_l40	2.426	2.211	2.189	2.186	2.250	2.203	2.331
h16_l15	1.873	1.904	1.878	1.878	1.913	1.934	1.963

Appendix B. Effects against static HEFT and blind B4, all workloads

TABLE XXIX

CPU only: effect on end-to-end makespan against static HEFT_param_model (%, negative = faster; exploratory except h16_l120)

Workload	B4 param-first	PRED	PRED-E2E
h16_l120	−4.8 [−5.4, −4.3]	−5.2 [−5.9, −4.6]	−4.6 [−5.2, −4.0]
uniform	−1.9 [−4.2, +0.8]	−0.2 [−1.2, +0.9]	−0.2 [−2.1, +2.3]
h04_l120	−1.4 [−3.3, +0.7]	−2.4 [−3.9, −0.6]	−2.7 [−4.4, −1.0]
h40_l120	−4.1 [−4.7, −3.5]	−4.8 [−5.4, −4.2]	−5.1 [−5.9, −4.3]
h16_l40	−2.1 [−3.1, −0.8]	−3.2 [−3.9, −2.3]	−1.1 [−2.5, +0.2]
h16_l15	−1.7 [−2.3, −1.2]	−1.4 [−1.9, −0.9]	−0.9 [−1.8, −0.1]

TABLE XXX

strict GPU set: effect on end-to-end makespan against static HEFT_param_model (%, negative = faster; exploratory except h16_l120)

Workload	B4 param-first	PRED	PRED-E2E
h16_l120	−12.1 [−13.0, −11.4]	−12.2 [−13.3, −11.1]	−12.3 [−13.3, −11.5]
uniform	−2.2 [−4.2, +0.0]	−3.7 [−4.7, −2.8]	−0.9 [−3.3, +2.3]
h04_l120	−14.1 [−15.3, −12.7]	−12.8 [−13.6, −12.2]	−13.2 [−14.0, −12.3]
h40_l120	−8.9 [−10.7, −7.1]	−8.6 [−10.7, −6.6]	−8.8 [−10.5, −7.1]
h16_l40	−17.1 [−17.9, −16.4]	−17.1 [−17.8, −16.4]	−17.7 [−18.3, −17.0]
h16_l15	−13.3 [−14.2, −12.1]	−12.2 [−13.2, −11.1]	−11.8 [−12.7, −10.7]

TABLE XXXI

extended GPU set: effect on end-to-end makespan against static HEFT_param_model (%, negative = faster; exploratory except h16_l120)

Workload	B4 param-first	PRED	PRED-E2E
h16_l120	−1.4 [−2.5, −0.4]	−2.0 [−3.1, −1.0]	−2.6 [−3.6, −1.6]
h04_l120	−5.6 [−6.7, −4.5]	−4.3 [−5.5, −3.0]	−4.6 [−5.6, −3.7]
h40_l120	−1.0 [−3.1, +1.2]	−1.5 [−2.2, −0.7]	−1.8 [−3.1, −0.4]
h16_l40	−1.8 [−2.1, −1.4]	−2.7 [−3.6, −1.7]	−2.9 [−3.5, −2.2]
h16_l15	−0.5 [−1.1, +0.1]	−1.8 [−2.7, −1.0]	−1.8 [−2.9, −0.8]

TABLE XXXII

CPU only: effect on end-to-end makespan against blind B4 (%, negative = faster; exploratory except h16_l120)

Workload	B4 param-first	PRED	PRED-E2E
h16_l120	−17.5 [−18.3, −16.6]	−17.8 [−18.8, −16.7]	−17.3 [−18.2, −16.4]
uniform	−1.7 [−5.1, +1.0]	+0.0 [−2.5, +2.3]	0.0 [−1.8, +1.7]
h04_l120	−38.7 [−41.8, −35.4]	−39.4 [−42.3, −36.3]	−39.6 [−42.8, −36.2]
h40_l120	−2.0 [−2.9, −1.2]	−2.8 [−3.4, −2.2]	−3.0 [−3.8, −2.4]
h16_l40	−3.8 [−6.5, −0.7]	−4.8 [−7.3, −2.4]	−2.9 [−5.8, +0.3]
h16_l15	−1.0 [−1.8, −0.1]	−0.7 [−1.5, +0.2]	−0.2 [−1.0, +0.6]

TABLE XXXIII

strict GPU set: effect on end-to-end makespan against blind B4 (%, negative = faster; exploratory except h16_l120)

Workload	B4 param-first	PRED	PRED-E2E
h16_l120	−21.0 [−22.2, −19.8]	−21.2 [−22.5, −19.7]	−21.2 [−22.5, −19.9]
uniform	+1.5 [−0.5, +3.9]	0.0 [−1.3, +1.1]	+2.9 [+0.4, +5.9]
h04_l120	−40.9 [−45.3, −36.1]	−40.1 [−44.5, −35.0]	−40.3 [−45.0, −34.9]
h40_l120	−6.3 [−8.0, −4.8]	−6.0 [−7.8, −4.2]	−6.1 [−7.7, −4.7]
h16_l40	−7.9 [−9.1, −6.7]	−7.9 [−9.2, −6.4]	−8.6 [−10.0, −7.1]
h16_l15	−0.2 [−1.4, +1.1]	+1.1 [+0.3, +1.9]	+1.5 [+0.6, +2.5]

TABLE XXXIV

EXTENDED GPU SET: EFFECT ON END-TO-END MAKESPAN AGAINST BLIND B4 (%, NEGATIVE = FASTER; EXPLORATORY EXCEPT H16_L120)

Workload	B4 param-first	PRED	PRED-E2E
h16_l120	−25.6 [−26.7, −24.4]	−26.0 [−27.3, −24.7]	−26.4 [−27.7, −25.2]
h04_l120	−31.0 [−38.4, −22.8]	−30.0 [−37.7, −21.5]	−30.3 [−37.8, −21.7]
h40_l120	−8.5 [−9.6, −7.4]	−9.0 [−10.5, −7.6]	−9.2 [−10.0, −8.4]
h16_l40	−8.9 [−11.1, −6.6]	−9.8 [−11.3, −8.2]	−9.9 [−11.4, −8.4]
h16_l15	+1.6 [+0.0, +3.3]	+0.3 [−1.2, +1.9]	+0.3 [−1.4, +1.8]

APPENDIX B2. PRED AGAINST THE HAND RULE B4_PARAM_FIRST

TABLE XXXV

PRED AGAINST B4_PARAM_FIRST ON THE PRIMARY WORKLOAD, END-TO-END MAKESPAN (%, NEGATIVE = PRED FASTER; EXPLORATORY, 24 BLOCKS)

Hardware set	Effect [95% CI]
CPU only	−0.4 [−0.9, +0.1]
strict GPU set	−0.2 [−0.9, +0.7]
extended GPU set	−0.6 [−1.1, 0.0]

APPENDIX C. PLACEMENT OF TASKS (PRIMARY WORKLOAD)

TABLE XXXVI

TASKS PER RUN ON CPU / GPU, GPU BUSY FRACTION AND MEAN CPU UTILISATION

Set	Policy	Tasks CPU / GPU	GPU busy	CPU util.
strict	B4 (blind, dynamic)	175 / 82	0.43	0.80
strict	B4 param-first (rule, dynamic)	224 / 33	0.27	0.96
strict	PRED (dynamic)	218 / 39	0.33	0.96
strict	PRED-E2E (dynamic)	218 / 39	0.33	0.96
strict	HEFT (static)	196 / 61	0.35	0.82
strict	HEFT-E2E (static)	207 / 50	0.31	0.86
strict	PEFT (static)	183 / 74	0.40	0.75
extended	B4 (blind, dynamic)	220 / 37	0.63	0.77
extended	B4 param-first (rule, dynamic)	225 / 32	0.65	0.96
extended	PRED (dynamic)	212 / 45	0.76	0.96
extended	PRED-E2E (dynamic)	212 / 45	0.77	0.96
extended	HEFT (static)	208 / 49	0.86	0.85
extended	HEFT-E2E (static)	227 / 30	0.80	0.85
extended	PEFT (static)	247 / 10	0.79	0.71

REFERENCES

[1] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[2] H. Arabnejad and J. G. Barbosa, “List scheduling algorithm for heterogeneous systems by an optimistic cost table,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014, doi: 10.1109/TPDS.2013.57.

[3] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, “StarPU: A unified platform for task scheduling on heterogeneous multicore architectures,” *Concurrency Comput.: Pract. Exper.*, 2011 (journal version; see also Euro-Par 2009, LNCS 5704, pp. 863–874).

[4] J. Luo, Y. Zhou, X. Li, M. Yuan, J. Zeng, and J. Yao, “Learning to optimize DAG scheduling in heterogeneous environment,” in *Proc. IEEE Int. Conf. Mobile Data Manage. (MDM)*, 2022, doi: 10.1109/MDM55031.2022.00040 (arXiv:2103.06980).

[5] X. Chen, J. Cao, Y. Sahni, S. Jiang, and Z. Liang, “Dynamic task offloading in edge computing based on dependency-aware reinforcement learning,” *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 594–608, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3381646.

[6] Q. Tan, W. Chen, and D. Liu, “Task scheduling for heterogeneous multicore processors based on deep reinforcement learning,” *Int. J. Intell. Syst.*, vol. 2025, Art. no. 7562400, 2025, doi: 10.1155/int/7562400.

[7] J. Liu, P. Xie, W. Li, B. Tang, and J. Liu, “Dynamic task offloading scheme for edge computing via meta-reinforcement learning,” *Comput. Mater. Contin.*, vol. 82, no. 2, 2025, doi: 10.32604/cmc.2024.058810.

[8] C. Liu, H. Wang, M. Zhao, J. Liu, X. Zhao, and P. Yuan, “Dependency-aware online task offloading based on deep reinforcement learning for IoV,” *J. Cloud Comput.*, vol. 13, Art. no. 136, 2024, doi: 10.1186/s13677-024-00701-0.

[9] Y. Song, C. Li, L. Tian, and H. Song, “A reinforcement learning based job scheduling algorithm for heterogeneous computing environment,” *Comput. Electr. Eng.*, vol. 107, Art. no. 108653, 2023, doi: 10.1016/j.compeleceng.2023.108653.

[10] C. Sun, T. Yang, and Y. Lei, “DDDQN-TS: A task scheduling and load balancing method based on optimized deep reinforcement learning in heterogeneous computing environment,” *Int. J. Intell. Syst.*, vol. 37, no. 11, pp. 9138–9172, 2022, doi: 10.1002/int.22983.

[11] J. Wang, J. Hu, G. Min, W. Zhan, A. Y. Zomaya, and N. Georgalas, “Dependent task offloading for edge computing based on deep reinforcement learning,” *IEEE Trans. Comput.*, 2022, doi: 10.1109/TC.2021.3131040.

[12] B. Gong and X. Jiang, “Dependent task-offloading strategy based on deep reinforcement learning in mobile edge computing,” *Wireless Commun. Mobile Comput.*, 2023, doi: 10.1155/2023/4665067.

[13] J. Liu, Y. Mi, X. Zhang, and X. Li, “Task graph offloading via deep reinforcement learning in mobile edge computing,” *Future Gener. Comput. Syst.*, vol. 158, pp. 545–555, 2024, doi: 10.1016/j.future.2024.04.034 (arXiv:2309.10569).

[14] Z. Cao, X. Deng, S. Yue, P. Jiang, J. Ren, and J. Gui, “Dependent task offloading in edge computing using GNN and deep reinforcement learning,” *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21632–21646, 2024, doi: 10.1109/JIOT.2024.3374969.

[15] G. Nieto, I. de la Iglesia, U. Lopez-Novoa, and C. Perfecto, “Deep reinforcement learning techniques for dynamic task offloading in the 5G edge-cloud continuum,” *J. Cloud Comput.*, vol. 13, Art. no. 94, 2024, doi: 10.1186/s13677-024-00658-0.

[16] A. Rodríguez, A. Navarro, K. Nikov, J. Nunez-Yanez, R. Gran, D. Suárez Gracia, and R. Asenjo, “Lightweight asynchronous scheduling in heterogeneous reconfigurable systems,” *J. Syst. Archit.*, vol. 124, Art. no. 102398, 2022, doi: 10.1016/j.sysarc.2022.102398.

[17] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, “Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures,” *Future Gener. Comput. Syst.*, vol. 150, pp. 171–185, 2024, doi: 10.1016/j.future.2023.08.022.

[18] A. Ilyushkin and D. Epema, “The impact of task runtime estimate accuracy on scheduling workloads of workflows,” in *Proc. IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput. (CCGrid)*, 2018, pp. 331–341, doi: 10.1109/CCGRID.2018.00048.

[19] T. Lu and S. Reda, “Agentic CPU-GPU scheduling for heterogeneous AI workloads,” arXiv:2607.22242, Jul. 2026.

[20] S. Nagpal, A. Karumannil, V. Jain, and A. K. Patil (Xilinx, Inc.), “Scheduling processing of machine learning tasks on heterogeneous compute circuits,” U.S. Patent 11,561,826 B1, Jan. 24, 2023 (filed Nov. 12, 2020).

[21] A. Vega *et al.* (International Business Machines Corp.), “Heterogeneous system on a chip scheduler with learning agent,” U.S. Patent 11,740,933 B2, Aug. 29, 2023.

[22] A. Amarnath *et al.* (International Business Machines Corp.), "Learning agent based application scheduling," U.S. Patent 11,966,776 B2, Apr. 23, 2024.

[23] R. Kim and M.-S. Kim (Samsung Electronics Co., Ltd.), "Method and apparatus for performing task scheduling in terminal," U.S. Patent 10,162,671 B2, Dec. 25, 2018.

[24] R. Barik, T. Shpeisman, B. T. Lewis, and R. Kaleem (Intel Corp.), "Adaptive scheduling for task assignment among heterogeneous processor cores," U.S. Patent 10,186,007 B2, Jan. 22, 2019.

[25] Intel, "What is Intel Thread Director?," Intel support article 000097053, https://www.intel.com/content/www/us/en/support/articles/000097053/processors.html.

[26] The Linux Kernel documentation, "Energy Aware Scheduling," https://docs.kernel.org/scheduler/sched-energy.html.

[27] Microsoft, "Quality of Service (Win32 apps)," Microsoft Learn, 2025, https://learn.microsoft.com/en-us/windows/win32/procthread/quality-of-service.

[28] The Kubernetes Authors, "Kubernetes Scheduler (kube-scheduler)," https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/.

[29] Amazon Web Services, "What is AWS Batch?," AWS Batch User Guide, https://docs.aws.amazon.com/batch/latest/userguide/what-is-batch.html.

[30] SchedMD, "Generic Resource (GRES) scheduling," Slurm Workload Manager documentation, https://slurm.schedmd.com/gres.html.

[31] Anyscale/Ray Project, "Scheduling — Ray Core documentation," https://docs.ray.io/en/latest/ray-core/scheduling/index.html.

[32] Caltech/IPAC, "Montage: An astronomical image mosaic engine," software, https://github.com/Caltech-IPAC/Montage (commit ac0a1533, used in this work).

[33] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, "WfCommons: A framework for enabling scientific workflow research and development," *Future Gener. Comput. Syst.*, vol. 128, pp. 16–27, 2022, doi: 10.1016/j.future.2021.09.043.

APPENDIX D. ENVIRONMENT AND COMMANDS

Environment. Intel Core i5-13500HX, NVIDIA GeForce RTX 3050 6 GB (driver 581.83), 15.7 GB RAM, Windows 11, Python 3.11.9, PyTorch 2.11 with the CUDA 12.8 build; sessions need mains power and at least 2.4 GB of free memory (0.8 GB per block). **Commands (PowerShell, repository root).** `scripts/final_stage1.ps1` (calibration sessions, models, designs, smoke test); `scripts/final_stage2.ps1` (six evaluation sessions and their analyses); `scripts/final_analyze.py <session> --confirmatory` (primary session); `scripts/final_results_table.py`, `scripts/final_cross_workload.py`, `scripts/final_hybrid_check.py`, `scripts/final_build_report.py`; tests: `python -m pytest -q`. Hashes of the protocol, the designs and the models are in `data/results/final_20261101/ACTIVITY_LOG.md`.